RESOLVELOOP

Engineering Implementation Plan

Two-person build specification | October 10 – 30, 2026

Build target. An evidence-gated autonomous incident-response agent. It reproduces an authorized checkout-service crash, generates and verifies a patch, challenges the repair with human-authored behavioral invariants, optionally fixes a second confirmed defect, and opens an auditable GitHub pull request.

Scope. One Python FastAPI target, pytest, a Nebius-hosted NVIDIA model (exact model ID verified at runtime), E2B, Tavily, GitHub API, SQLite, and a lightweight event timeline. This is an implementation specification and interface contract, not finished production software. File names, APIs, SQL schema and sprint order are proposed engineering decisions.

Contents

1. Definition of done and strict claims . . . 2
2. Team ownership and merge contracts . . . 2
3. Repository tree: every tracked file . . . 3
4. Canonical data models and fields . . . 5
5. Configuration and dependencies . . . 6
6. Function-by-function specification . . . 6
7. Trusted contract design: oracle semantics . . . 10
8. Incident schema and example . . . 11
9. State machine and execution algorithm . . . 11
10. Independent verifier and threat model . . . 12
11. Test matrix: exact pass/fail cases . . . 13
12. Six-incident evaluation plan . . . 13
13. Delivery schedule: two parallel lanes . . . 14
14. Setup-to-demo command checklist . . . 14
15. PR and evidence artifact specification . . . 14
16. Three-minute demo storyboard . . . 15
17. Definition-of-done checklist . . . 15
18. Engineering decisions to verify before coding . . . 15
19. Handoff order: first five pull requests . . . 16

1. Definition of done and strict claims

Core success. Alert accepted → pinned baseline cloned → provided reproducer fails → model produces a permitted patch → reproducer passes → original suite passes → evidence artifact produced → human-review PR opened. If any gate fails, there is no VERIFIED claim.

Bonus success. A model proposes a structured scenario; the trusted invariant runner confirms an independently reproducible new violation; the agent repairs it; baseline, invariant and regression checks pass. If this is unreliable by **Oct 20**, label discovery EXPERIMENTAL and demo the core.

Explicit limits

- Stack traces alone do not guarantee reproduction.
- Seeded bugs are disclosed.
- Passing tests are bounded evidence, not mathematical proof.
- The system never merges or deploys changes.

2. Team ownership and merge contracts

The work is split in half by effort and by difficulty. Each person owns one hard lane and one easier lane, so nobody carries all the critical parts. The two hardest integration points (the sandbox and the verifier) are built as a pair: one writes it, the other reviews and tests it.

Owner	Responsibilities	Delivery contract
Satyabrata Builder A: AI agent + verification	Day-one model test, Nebius client, prompts, bounded context, patch loop, baseline + final verifier, orchestrator, scenario proposals, GitHub PR step, progress UI	Owns `agent/ (nebius, prompts, context, patcher) orchestrator.py verifier/ (baseline, verify) discovery/scenarios.py github/ ui/`
Atina Builder B: sandbox + trusted evidence + app	Checkout demo app, invariants, signed webhook, store, E2B sandbox layer, patch policy, evidence manifest, trusted scenario runner + recheck, Tavily search, post-mortem, docs and video	Owns `demo_app/ api/ store.py security.py events.py sandbox/ verifier/ (patch_policy, evidence) discovery/ (catalog, runner, recheck) agent/research.py reporting/ docs/`
Both	Shared schema, CI, security review, blind tests, rehearsal, evaluation runs, demo video	Daily 15-minute sync; one pull request per isolated feature; each person reviews the other's PRs before merge

> **Interfaces frozen on day one:** IncidentEvidence, RunEvent, PatchCandidate, VerificationResult, ScenarioSpec, DiscoveryResult, RunResult. Atina supplies canned incident JSON and the buggy app; Satyabrata supplies canned RunResult JSON. Neither lane waits for the other.

Why this is balanced

- **Satyabrata:** the model-facing work (unpredictable) plus the orchestrator that joins everything.
- **Atina:** the safety-critical, deterministic work (sandbox, patch policy, evidence hashes, trusted oracle runner). It is the trust core of the project and is testable with plain pytest, with no model behavior involved.
- Both lanes have one piece that is demo-visible: Satyabrata the live progress UI and PR, Atina the checkout app, the evidence file and the post-mortem.

Satyabrata: pieces, in order

#	Piece	Done when
A1	Day-one test: can a Nemotron model fix the seeded bug, and from the stack trace alone? Plus model probe	Result written down; decides loop effort
A2	Nebius client: chat calls, retries, timeouts, token limits (`agent/nebius.py`)	Healthcheck lists the verified model ID
A3	Prompts and bounded context (`agent/prompts.py`, `agent/context.py`)	Prompt returns file/line evidence on a sample trace
A4	Baseline and final verifier (`verifier/baseline.py`, `verifier/verify.py`)	Red baseline then green; hashes unchanged; suite green
A5	Patch loop and orchestrator (`agent/patcher.py`, `orchestrator.py`)	Alert to verified patch within 3 attempts
A6	Scenario proposals from the model (`discovery/scenarios.py`) and optional second repair	Go / no-go on Oct 20; otherwise labeled experimental
A7	GitHub PR step and progress UI (`github/`, `ui/`)	PR opens on the demo repo from one command; UI plays a recorded run

Atina: pieces, in order

#	Piece	Done when
B1	Checkout demo app with pytest suite: one crash bug with its failing test, one hidden second bug, and a control copy with no second bug (`demo_app/`)	pytest shows exactly the planned failure; a one-line fix makes it pass; control copy clean
B2	Trusted rules and scenario catalog (`contracts/`, `discovery/catalog.py`)	Each rule fails on the buggy app and passes on the control
B3	Signed webhook, traceback parser, SQLite store, events, redaction (`api/`, `store.py`, `security.py`, `events.py`)	One command triggers a run; invalid signature returns 401
B4	E2B sandbox layer: create, pinned checkout, fixed commands, destroy (`sandbox/`)	Clone at SHA, run pytest, destroy; run on a hello-world repo first
B5	Patch policy and evidence manifest (`verifier/patch_policy.py`, `verifier/evidence.py`)	Diffs touching tests/contracts are rejected; manifest hashes verify
B6	Trusted scenario runner, repeat confirmation, negative control (`discovery/runner.py`, `recheck.py`)	Violation confirmed twice on buggy app; none on control
B7	Tavily search and post-mortem template (`agent/research.py`, `reporting/`)	Three cited results; sample run produces a clean PR body
B8	README, evaluation CSV, feedback log (`docs/`)	Fresh machine can run the demo from the README

Working rules: small pull requests; each person reviews the other's PRs; stuck for more than two hours means ask the other. B4 and A4 are the two hardest integration points, so book a short pairing session for each. Out of the MVP: arbitrary model-written test files, automated code review, multi-agent debate.

3. Repository tree: every tracked file

Owner key: A = Satyabrata, B = Atina, Both = shared. Paths are relative to the repository root.

Area	Path	Owner	Purpose
ROOT	`.gitignore`	Both	Ignore secrets, caches, sandbox output
ROOT	`.env.example`	B	Configuration variable names only
ROOT	`pyproject.toml`	A	Dependencies, pytest, Ruff config
ROOT	`README.md`	B	Setup, demo, architecture, honest limitations
ROOT	`LICENSE`	B	MIT / Apache-2.0
ROOT	`Makefile`	A	Local install, lint, test, run, demo

Area	Path	Owner	Purpose
ROOT	`.github/workflows/ci.yml`	B	Unit tests, lint, secret scanning
CORE	`resolveloop/config.py`	A	Settings validation
CORE	`resolveloop/models.py`	Both	Pydantic interfaces and enums
CORE	`resolveloop/store.py`	B	SQLite run/event persistence
CORE	`resolveloop/security.py`	B	Webhook HMAC, redaction, allowlist
CORE	`resolveloop/events.py`	B	Event append and stream helper
CORE	`resolveloop/orchestrator.py`	A	State machine and gates
API	`resolveloop/api/main.py`	B	FastAPI startup / routes
API	`resolveloop/api/webhooks.py`	B	Signed alert intake
API	`resolveloop/api/runs.py`	B	Run status, event stream, evidence endpoints
AGENT	`resolveloop/agent/nebius.py`	A	Inference wrapper, retries, token limits
AGENT	`resolveloop/agent/prompts.py`	A	Diagnosis, patch, scenario prompt templates
AGENT	`resolveloop/agent/context.py`	A	Bounded repo / context gathering
AGENT	`resolveloop/agent/patcher.py`	A	Diff proposals and bounded repair loop
AGENT	`resolveloop/agent/research.py`	B	Tavily search, citations, sanitization
SANDBOX	`resolveloop/sandbox/e2b_runner.py`	B	Ephemeral sandbox lifecycle
SANDBOX	`resolveloop/sandbox/repository.py`	B	Pinned checkout, guarded read/write paths
SANDBOX	`resolveloop/sandbox/commands.py`	B	Fixed allowlisted commands
VERIFY	`resolveloop/verifier/baseline.py`	A	Before-patch red test + hashes
VERIFY	`resolveloop/verifier/patch_policy.py`	B	Diff safety validation
VERIFY	`resolveloop/verifier/verify.py`	A	After-patch tests and evidence
VERIFY	`resolveloop/verifier/evidence.py`	B	JSON manifest + hashes
DISCOVERY	`resolveloop/discovery/scenarios.py`	A	Model-generated typed scenario proposals
DISCOVERY	`resolveloop/discovery/catalog.py`	B	Trusted scenario IDs, invariant registry
DISCOVERY	`resolveloop/discovery/runner.py`	B	Execute scenario and check oracle
DISCOVERY	`resolveloop/discovery/recheck.py`	B	Repeat confirmation and negative controls
OUTPUT	`resolveloop/reporting/postmortem.py`	B	PR report generator
OUTPUT	`resolveloop/github/client.py`	A	Branch, commit, PR create
OUTPUT	`resolveloop/ui/index.html`	A	Simple progress page
OUTPUT	`resolveloop/ui/app.js`	A	Poll events and render states
OUTPUT	`resolveloop/ui/style.css`	A	Minimal dashboard style
DEMO	`demo_app/pyproject.toml`	B	Pinned app dependencies
DEMO	`demo_app/app/main.py`	B	FastAPI checkout routes
DEMO	`demo_app/app/models.py`	B	Request / response types
DEMO	`demo_app/app/payments.py`	B	Mock payment provider
DEMO	`demo_app/app/store.py`	B	In-memory order / charge records
DEMO	`demo_app/tests/test_baseline.py`	B	Original suite (immutable snapshot)
DEMO	`demo_app/tests/test_incident.py`	B	Known failing reproducer
DEMO	`demo_app/contracts/invariants.json`	B	Human-authored behavioral contracts

Area	Path	Owner	Purpose
DEMO	`demo_app/fixtures/incident_01.json`	B	Alert with pinned SHA and fixture
DEMO	`demo_app/fixtures/scenario_cases.json`	B	Trusted scenario templates
DEMO	`demo_app/seed_variants/README.md`	B	Disclose seeded variants and hidden bugs
TESTS	`tests/test_security.py`	B	HMAC, allowlist, redaction
TESTS	`tests/test_models.py`	Both	Schema compatibility
TESTS	`tests/test_orchestrator.py`	A	State machine and stop cases
TESTS	`tests/test_verifier.py`	A	Red/green and immutable guard
TESTS	`tests/test_discovery.py`	B	True / false scenario detection
TESTS	`tests/test_github.py`	A	PR mocked idempotence
TESTS	`tests/test_api.py`	B	Webhook + status contract
DOCS	`docs/demo_script.md`	B	3-minute storyboard
DOCS	`docs/evaluation.csv`	Both	Real repeated outcomes
DOCS	`docs/feedback.md`	Both	Honest Nebius / NVIDIA feedback
DOCS	`docs/architecture.md`	A	Trust boundaries, sequence and ADRs

4. Canonical data models and fields

Model	Fields	Rule
RunStatus	`RECEIVED, BASELINING, UNREPRODUCED, DIAGNOSING, PATCHING, VERIFYING, CHALLENGING, CONFIRMED_DISCOVERY, VERIFIED, FAILED, PR_OPENED`	State machine enum
IncidentEvidence	`incident_id:str; repo_url:str; commit_sha:str; error_type:str; message:str; traceback:str; reproducer_id:str; sanitized_request:dict|None; timestamp:datetime; environment:dict`	Validate repo allowlist, exact SHA, max payload size; never accept an arbitrary shell command
RunEvent	`run_id:str; seq:int; stage:str; severity:str; message:str; created_at:datetime; metadata:dict`	Redacted before storage and UI
PatchCandidate	`base_sha:str; unified_diff:str; rationale:str; attempt:int; touched_paths:list[str]`	No test / contracts / CI edits
CommandResult	`argv:list[str]; exit_code:int; stdout:str; stderr:str; duration_ms:int; truncated:bool`	Fixed argv; capture sanitized output
BaselineResult	`reproducer_failed:bool; original_suite:dict; baseline_sha:str; contract_hash:str; test_hashes:dict`	Baseline must fail for the intended reason
VerificationResult	`baseline_red:bool; reproducer_green:bool; suite_green:bool; paths_safe:bool; oracle_intact:bool; test_counts:dict; status:str`	Verifier, not the LLM, sets status
ScenarioSpec	`scenario_id:str; invariant_id:str; params:dict; rationale:str`	Constrained enum, strict schema; no code or expected result generated
DiscoveryResult	`scenario:ScenarioSpec; oracle_id:str; observed:dict; violated:bool; reproduced_twice:bool; baseline_commit:str`	Confirmed only when a trusted invariant is violated reproducibly
RunResult	`run_id:str; status:RunStatus; incident:IncidentEvidence; patch:PatchCandidate|None; verification:VerificationResult|None; discoveries:list; events:list; evidence_path:str|None; pr_url:str|None`	One durable object consumed by reporting / UI

5. Configuration and dependencies

```
NEBIUS_API_KEY=
NEBIUS_BASE_URL=
NEBIUS_MODEL_ID=
TAVILY_API_KEY=
E2B_API_KEY=
GITHUB_TOKEN=
GITHUB_OWNER=
GITHUB_TARGET_REPO=
WEBHOOK_SECRET=
ALLOWED_REPO_URL=
DATABASE_URL=sqlite:///./resolveloop.db
MAX_PATCH_ATTEMPTS=3
MAX_SCENARIOS=4
SANDBOX_TIMEOUT_SECONDS=180
RUN_DISCOVERY=true
OPEN_PR_ON_VERIFIED=true
```

config.py: `class Settings(BaseSettings)` holding every variable above. Validate max attempts in 1..5, max scenarios in 0..5, strictly non-empty secrets, and an exact allowlisted repository. Never log keys. Functions: `get_settings()->Settings`, `validate_startup(settings)->None`, `public_config()->dict` (no secrets).

pyproject.toml: Python 3.11+; fastapi, uvicorn, pydantic, pydantic-settings, httpx, pytest, pytest-asyncio, an OpenAI-compatible SDK, tavily-python, the E2B SDK (verify current import/API), PyGithub or direct httpx, ruff. Pin tested versions in a lockfile. Do not blindly copy SDK method names from this plan.

6. Function-by-function specification

resolveloop/security.py

`verify_webhook_signature(body:bytes, signature:str, secret:str)->bool`
HMAC-SHA256 with constant-time comparison; reject missing or invalid signature.

`assert_allowed_repo(repo_url:str, sha:str, settings:Settings)->None`
Exact URL allowlist and 40-character hex SHA; never allow arbitrary repos.

`redact_text(value:str)->str`
Remove tokens, authorization headers, email and payment data before anything reaches the model or events.

resolveloop/store.py

`init_db()->None`
Create runs, events, evidence tables with unique run_id and incident_id.

`create_run(incident:IncidentEvidence)->str`
Idempotent insert by incident ID; return the existing run if duplicate.

`set_run_status(run_id:str, status:RunStatus)->None`
Validate permitted transition; persist timestamp.

`append_event(event:RunEvent)->None`
Monotonic sequence per run; transactional write.

`get_run(run_id:str)->RunResult|None`
Return redacted public run state.

resolveloop/events.py

`emit(run_id:str, stage:str, message:str, metadata:dict={})->RunEvent`
Redact, timestamp, persist. The event log is the source of truth.

`list_events(run_id:str, after_seq:int=0)->list[RunEvent]`
Stable order for the polling UI.

resolveloop/api/main.py

`create_app()->FastAPI`
Register routes, startup settings/db, static UI. No network side effects at import.

resolveloop/api/webhooks.py

`POST /webhooks/incident`
Verify HMAC over the raw body; validate evidence; deduplicate; enqueue run; respond 202 + run_id.

`parse_alert(payload:dict)->IncidentEvidence`
One supported Python traceback format; reject unknown reproducer ID.

resolveloop/api/runs.py

`GET /runs/{run_id}`
Return public status and summary.

`GET /runs/{run_id}/events?after=0`
Return the append-only event list.

`GET /runs/{run_id}/evidence`
Return sanitized evidence JSON if authorized.

resolveloop/orchestrator.py

`run_incident(incident:IncidentEvidence)->RunResult`
Execute the exact gate sequence; catch errors; persist FAILED with a reason.

`advance(run_id:str, next_status:RunStatus)->None`
Reject illegal transitions.

`maybe_create_pr(result:RunResult)->RunResult`
Only after VERIFIED; fail closed if checks are missing.

resolveloop/agent/nebius.py

`generate_json(system:str, user:str, schema:type, timeout_s:int)->dict`
Model call; parse against schema; retry transport errors only; bounded context.

`healthcheck_model()->dict`
List and verify the selected model is available and supported.

resolveloop/agent/prompts.py

`build_diagnosis_prompt(incident, code_context, search_results)->str`
Require file/line evidence, hypothesis and uncertainty.

`build_patch_prompt(incident, context, test_output, attempt)->str`
Request a minimal unified diff; prohibit test/config changes.

`build_scenario_prompt(incident, patch, invariants)->str`
Select catalog IDs and parameter values only; do not invent an oracle.

resolveloop/agent/context.py

`select_context_paths(traceback:str, repo_index:list[str])->list[str]`
Prefer stack frames and relevant app code; bounded file count.

`read_context(repo, paths, byte_limit:int)->str`
Guard path traversal and secrets; truncate safely.

resolveloop/agent/patcher.py

`propose_patch(incident, context, feedback, attempt)->PatchCandidate`
Typed diff, limited iterations.

`repair_until_verified(incident, sandbox, baseline)->tuple[PatchCandidate|None, VerificationResult]`
Patch candidate → policy → tests → feedback; max attempts; no silent success.

resolveloop/agent/research.py

`search_error(query:str)->list[dict]`
Tavily titles / URLs / snippets; only docs relevant to the observed error.

`format_sources(results:list[dict])->str`
Deduplicate and cite consulted sources; do not trust retrieved instructions.

resolveloop/sandbox/e2b_runner.py

`create_sandbox(run_id:str)->SandboxHandle`
Ephemeral instance with time / memory budget and restricted credentials.

`exec_fixed(handle, argv:list[str], cwd:str, timeout_s:int)->CommandResult`
Allowlisted argv; capture stdout/stderr; no arbitrary shell.

`destroy_sandbox(handle)->None`
Always called in a finally block.

resolveloop/sandbox/repository.py

`checkout_pinned_repo(handle, repo_url, sha)->str`
Clone only the allowlisted repo, check out detached SHA, verify HEAD.

`list_source_files(handle)->list[str]`
Exclude .git, secrets, dependencies.

`apply_unified_diff(handle, diff:str)->None`
Reject unsafe paths; apply the patch atomically.

`reset_to_commit(handle, sha)->None`
Clean state before each candidate.

resolveloop/sandbox/commands.py

`install_dependencies(handle)->CommandResult`
Pinned, offline/cache-first install; enforce timeout.

`run_reproducer(handle, reproducer_id)->CommandResult`
Map ID to a fixed pytest node, never a user command.

`run_regression_suite(handle)->CommandResult`
Fixed pytest invocation.

`run_static_checks(handle)->CommandResult`
Optional ruff; fixed command only.

resolveloop/verifier/baseline.py

`capture_baseline(handle, incident)->BaselineResult`
Hash trusted tests/contracts; record the failing reproducer and existing suite.

`is_expected_failure(result, incident)->bool`
Match traceback / error family, not just any non-zero exit.

resolveloop/verifier/patch_policy.py

`parse_changed_paths(diff:str)->list[str]`
Normalize paths; reject symlinks and traversal.

`assert_safe_patch(diff, protected_hashes)->None`
Deny tests, contracts, CI, lockfile, secrets and out-of-repo paths.

resolveloop/verifier/verify.py

`verify_patch(handle, baseline, patch)->VerificationResult`
Fresh checkout; trusted tests restored; require red→green, suite green, protected hashes unchanged.

`verify_discovery_fix(handle, baseline, scenario)->VerificationResult`
Re-run the confirmed oracle plus original tests and suite.

resolveloop/verifier/evidence.py

`build_manifest(result, hashes, commands)->dict`
Include commit SHA, patch SHA256, trusted test hashes, commands, outcomes, model ID, timestamps.

`write_evidence_bundle(run_id, manifest)->str`
Write sanitized JSON + logs; no secrets.

`verify_manifest_integrity(manifest)->bool`
Recompute hashes; reject mismatches.

resolveloop/discovery/catalog.py

`load_invariants(path)->dict`
Human-authored, version-pinned oracle IDs and descriptions.

`validate_scenario(spec:ScenarioSpec, catalog:dict)->None`
Strict ID / param schema, ranges, no arbitrary code.

resolveloop/discovery/scenarios.py

`propose_scenarios(incident, patch, catalog, max_count)->list[ScenarioSpec]`
LLM selects candidate catalog scenarios; deterministic dedupe and validation.

resolveloop/discovery/runner.py

`execute_scenario(handle, spec)->dict`
Dispatch only the trusted runner by scenario ID.

`evaluate_invariant(spec, observed, catalog)->bool`
Trusted Python assertion, not LLM judgment.

resolveloop/discovery/recheck.py

`confirm_violation(handle, spec)->DiscoveryResult`
Reset fixture; reproduce twice; capture exact observations.

`run_negative_control(handle, spec)->bool`
Verify no false discovery on a known-good variant.

resolveloop/reporting/postmortem.py

`render_pr_body(result:RunResult)->str`
Root cause, diff summary, red/green proof, scenario evidence, limitations, citations.

`render_failure_report(result)->str`
Honest UNREPRODUCED / FAILED outcome.

resolveloop/github/client.py

`create_branch(base_sha, run_id)->str`
Deterministic unique branch; based on the pinned SHA.

`commit_patch(branch, diff, manifest)->str`
Commit only verified source paths; no tokens.

`open_pull_request(branch, body)->str`
Human review; idempotent lookup before create; never auto-merge.

resolveloop/ui/app.js

`loadRun(runId)`
Fetch status and event log.

`renderTimeline(events)`
Display stage, time, pass/fail and sanitized metadata.

`renderEvidence(summary)`
Link to PR and evidence; never fabricate a green status.

demo_app/app/main.py

`POST /checkout`
Call the mock payment gateway; return a consistent order status.

`GET /orders/{order_id}`
Expose the stored order (for tests only).

demo_app/app/payments.py

`charge(amount, idempotency_key, mode)->PaymentResult`
Deterministic fake gateway: normal, malformed, timeout, retry.

`get_charge_count(key)->int`
Trusted observable for the duplicate-charge invariant.

demo_app/app/store.py

`create_order(order_id, status)->Order`
Record the order in a deterministic in-memory store.

`reset_state()->None`
Clear charges / orders before every scenario.

demo_app/tests/test_incident.py

`test_checkout_malformed_response_reproducer()`
Known red test at the pinned baseline; tests expected error behavior.

demo_app/tests/test_baseline.py

`test_success_creates_one_order()`
Baseline behavior.

`test_failed_payment_not_completed()`
Baseline behavior.

`test_valid_request_returns_response()`
Baseline behavior.

7. Trusted contract design: oracle semantics

Human-written invariants are versioned separately from agent-writable source. They are interpreted by trusted runner code. The model can select scenario IDs and bounded parameters, but it cannot author assertions or edit contracts.

```
{
  "version": "1.0",
  "invariants": {
    "at_most_one_charge":           {"scenario_ids": ["retry_same_key"], "max_attempts": 3},
    "failed_payment_not_completed": {"scenario_ids": ["gateway_decline"]},
    "success_exactly_one_order":    {"scenario_ids": ["normal_success"]}
  }
}
```

- **at_most_one_charge:** the same idempotency key with the same logical payment request must yield at most one charge.
- **failed_payment_not_completed:** a declined payment must not mark an order completed.
- **success_exactly_one_order:** a successful request must produce exactly one completed order.
- Define how the fake gateway exposes charge counts and how state is reset. Do not assert unspecified behavior (for example, that every timeout must return one particular HTTP status).

Scenario catalog

Scenario ID	Parameters	Notes
`retry_same_key`	attempts: 2..3	Maps to trusted Python test logic
`gateway_decline`	decline_code: approved enum	Maps to trusted Python test logic
`normal_success`	amount: 1..1000	Maps to trusted Python test logic
`malformed_gateway_response`	payload_variant: approved enum	Maps to trusted Python test logic

Unexpected scenario IDs are rejected.

8. Incident schema and example

```
{
  "incident_id": "demo-001",
  "repo_url": "https://github.com/ORG/checkout-demo",
  "commit_sha": "<EXACT_40_HEX_COMMIT_SHA>",
  "error_type": "TypeError",
  "message": "Unexpected payment response",
  "traceback": "Traceback (most recent call last): ...",
  "reproducer_id": "checkout_malformed_response",
  "sanitized_request": {"order_id": "order-123", "amount": 50},
  "environment": {"provider_mode": "malformed"},
  "timestamp": "2026-10-10T12:00:00Z"
}
```

The `reproducer_id` is mapped in trusted code to a known pytest node. It is **not** a path or shell command supplied by a webhook caller. In the demo, the failing test ships with the target repo and this is explicitly disclosed. Real production alerts would need additional evidence to construct a reproducer.

9. State machine and execution algorithm

```
RECEIVED -> BASELINING -> DIAGNOSING -> PATCHING -> VERIFYING
   |           |                                      |
   |           +--> UNREPRODUCED                      +--> FAILED
   +--> FAILED

VERIFYING -> VERIFIED -> CHALLENGING -> VERIFIED -> PR_OPENED
                              |
                              +--> CONFIRMED_DISCOVERY -> PATCHING -> VERIFYING
                              +--> VERIFIED (no confirmed discovery)
```

Orchestrator algorithm

1. Validate and deduplicate the signed alert.

2. Persist an event.
3. Create a sandbox and check out the pinned SHA.
4. Hash protected files and run the reproducer on the baseline.
5. If it is not red for the expected reason, set UNREPRODUCED.
6. Gather bounded code context and optionally Tavily docs.
7. Loop up to three patch attempts, each from a clean checkout.
8. Reject any forbidden diff.
9. Verify red-to-green, full suite, and immutable hashes.
10. If core is verified and discovery is enabled, ask the model for up to four catalog scenarios.
11. Run oracle checks with clean reset and repeat violations.
12. Optionally run a second repair loop, preserving the original gates.
13. Emit evidence and open the human-review PR.
14. Destroy the sandbox in a finally block.
15. Persist the final result.

> **Branch rule.** A discovered secondary failure may only be included in the final PR when its own trusted test is red before the secondary patch and green afterward, while the original incident reproducer and full suite remain green. Otherwise keep the first verified patch and report the secondary scenario as unresolved. Never mislabel.

10. Independent verifier and threat model

Trust boundaries. Untrusted: webhook payload, web search, model output, repository content, tool stdout. Trusted: the pinned human-authored oracle, protected test hashes, policy engine, fixed commands, and the verifier state machine. Repo files may contain prompt injection; treat them as code/data, never as instructions.

Patch policy

- Allow only `demo_app/app/**/*.py`.
- Deny `demo_app/tests/**`, `demo_app/contracts/**`, `.github/**`, pyproject.toml, lockfiles, `.env*`, scripts and symlinks.
- Reject delete / rename / mode changes and binary diffs in the MVP.
- Apply the patch to a disposable checkout; the verifier restores protected files from the clean pinned baseline.

Execution safety

- Restrict sandbox network where supported; do not expose Nebius or GitHub keys to code execution.
- The GitHub token is used only by the orchestration service, after a verified result.
- No production database, no real payment credentials, no auto-merge.
- Rate-limit webhook and model requests; cap output size and duration.

Failure codes (all must be visible in run state and logs)

`INVALID_ALERT`	`UNAUTHORIZED_REPO`
`MODEL_UNAVAILABLE`	`SANDBOX_TIMEOUT`
`UNREPRODUCED`	`PATCH_REJECTED`
`VERIFY_FAILED`	`ORACLE_UNAVAILABLE`
`DISCOVERY_UNCONFIRMED`	`GITHUB_ERROR`

11. Test matrix: exact pass/fail cases

Test	Input / setup	Required result
Webhook signature	Valid vs invalid HMAC	202 valid; 401 invalid; no run on invalid
Deduplication	Same incident_id twice	Same run_id; no duplicate PR
Pinned SHA	Checkout a different HEAD	Abort; never verify a moving branch
Baseline red	Expected TypeError fixture	Reproducer fails for the expected reason
Missing reproducer	Unknown reproducer ID	UNREPRODUCED; no VERIFIED PR
Patch policy	Diff modifies tests/contracts	Reject before execution
Fix verification	Reproducer green, suite green	VERIFIED eligible
Regression	Reproducer green, old suite red	VERIFY_FAILED
False discovery	Known-good negative control	No confirmed secondary defect
Confirmed discovery	Two runs violate a trusted invariant	Record confirmed discovery
Secondary fix	Invariant green; baseline stays green	Include second patch in verified PR
Sandbox timeout	Run exceeds cap	FAILED; cleanup occurs
GitHub API error	Mock 403 / 500	Preserve verified evidence; no false PR_OPENED

12. Six-incident evaluation plan

Use six controlled incidents/variants, with at least one negative control and one blind variant not used during prompt tuning. Do not claim these represent all production bugs. Repeat the headline demo several times and record failures as well as successes.

Case	Type	What to measure
01 malformed gateway response	Reported incident	Red→green and regression suite
02 duplicate retry charge	Secondary invariant defect	Unprompted scenario selection and oracle confirmation
03 declined payment order	Invariant family	Confirmed vs rejected hypothesis
04 inventory missing	Reported incident	Agent patch success / abstention
05 gateway timeout	Reported incident	Timeout handling and test boundaries
06 known-good variant	Negative control	False-positive count; should report no extra defect

`docs/evaluation.csv` columns: `case_id, variant_sha, run_id, seeded, blind, reproduced, patch_attempts, primary_verified, scenarios_proposed, secondary_confirmed, false_positive, pr_created, elapsed_seconds, failure_reason, evidence_hash`. Fill only from executed runs. Report numerator / denominator, not cherry-picked percentages.

13. Delivery schedule: two parallel lanes

Dates	Satyabrata (A)	Atina (B)	Shared acceptance gate
Oct 10–11	Nebius model probe, day-one test, models.py	Checkout app + failing test, E2B hello-world	Both run smoke tests; schemas frozen
Oct 12–14	Prompts, context, baseline verifier	Signed webhook, store, invariants, pinned clone in E2B	First red baseline in E2B
Oct 15–17	Patch loop, final verifier, orchestrator	Patch policy, evidence manifest, post-mortem	End-to-end alert → verified patch; PR step
Oct 18–20	Model scenario proposals, GitHub PR + UI	Trusted scenario runner, recheck, negative control	Go / no-go on autonomous discovery
Oct 21–23	Secondary patch if stable; reruns	6-case evaluation, CSV, docs	Feature freeze Oct 23
Oct 24–28	Hardening, rate limits, hosted endpoint	Demo recording, README, submission prep	Rehearsed video + real PR + evidence
Oct 29–30	Emergency fixes only	Submission review	Submit before the deadline; no last-minute features

14. Setup-to-demo command checklist

```
# 1. Create repository and virtual environment
python3.11 -m venv .venv
source .venv/bin/activate
pip install -e '.[dev]'
cp .env.example .env          # fill locally; never commit

# 2. Validate dependencies and models
python -m pytest tests/test_models.py tests/test_security.py
python -m resolveloop.scripts.model_probe        # implement first

# 3. Run checkout app and tests
python -m pytest demo_app/tests/test_baseline.py
python -m pytest demo_app/tests/test_incident.py   # expected RED on seeded baseline

# 4. Start orchestrator API
uvicorn resolveloop.api.main:app --reload --port 8000

# 5. Trigger demo through signed alert simulator
python -m resolveloop.scripts.send_demo_alert --fixture demo_app/fixtures/incident_01.json

# 6. Inspect run and PR
#    GET /runs/{run_id}   ;   GET /runs/{run_id}/events
```

> **Note.** `model_probe.py` and `send_demo_alert.py` do not exist yet. Write them as small wrappers before using commands 2 and 5. Validate the real package and import paths during day-one setup.

15. PR and evidence artifact specification

PR title: `[ResolveLoop] Verified repair for incident demo-001`

PR sections

- Incident summary
- Pinned baseline SHA
- Reproduced failing test (command, error)
- Root-cause hypothesis
- Changed paths and patch attempts
- Verified green tests and protected hashes

- Additional scenarios (confirmed / unconfirmed)
- Residual risks
- Tavily sources actually consulted
- Link to the JSON manifest

No assertion of guaranteed correctness.

Evidence manifest

```
{
  "run_id": "run-...",
  "baseline_sha": "<sha>",
  "patched_commit_sha": "<sha>",
  "patch_sha256": "<sha256>",
  "protected_test_sha256": {"tests/test_incident.py": "<sha256>"},
  "baseline_reproducer": {"exit_code": 1, "matched_expected": true},
  "patched_reproducer": {"exit_code": 0},
  "regression_suite": {"exit_code": 0, "passed": 8},
  "discovery": {"confirmed": false, "scenarios_run": 4},
  "model_id": "<verified-nebius-model-id>",
  "verified_at": "<utc-timestamp>",
  "limitations": ["Controlled seeded checkout demo"]
}
```

16. Three-minute demo storyboard

Time	Visual	Judge takeaway
0:00–0:20	Checkout crash + signed alert	Real trigger, no manual prompt
0:20–0:55	Pinned baseline failing, agent diagnosis	Reproduction is evidence, not assumption
0:55–1:35	Patch attempt and independent green verifier	The model cannot grade its own work
1:35–2:15	Scenario challenge; show confirmed defect if repeatable	Distinctive incident-to-immunity capability
2:15–2:40	Evidence manifest and GitHub PR	Auditable, human-reviewable result
2:40–3:00	6-case results, limitations, impact	Credible evaluation over hype

> **Disclosure line.** “This controlled checkout app contains seeded defects. ResolveLoop receives the first incident, while the second defect is not identified in the agent prompt. All findings are checked against prewritten requirements.” If discovery is not reliable, show it as experimental and spend that time on red/green verification and negative controls.

17. Definition-of-done checklist

Priority	Items
P0 must ship	Signed alert; exact allowlisted SHA; deterministic red baseline; no agent test editing; three-attempt bounded repair; clean verifier; evidence JSON; real GitHub PR; reproducible README; secrets scan
P1 should ship	Catalog-based scenarios; independent oracle; repeat confirmation; negative control; honest failure states; polished timeline
P2 if time	Second repair loop; incident memory; more variants

Never trade away P0 to chase P2.

18. Engineering decisions to verify before coding

These details can change, and this plan intentionally avoids inventing confirmed provider-specific method signatures. Verify each one and record the answer in `docs/architecture.md`.

Item to verify	What to confirm
Nebius Token Factory	Current base URL; actual available NVIDIA model ID; context limits; rate limits; pricing
E2B	SDK API and import paths; network restrictions
Tavily	Quota; citation fields
GitHub	Fine-grained token permissions; API semantics
Hackathon rules	Submission rules; hosted demo requirements; deadline and timezone on the official page

19. Handoff order: first five pull requests

PR	Title	Satyabrata (A)	Atina (B)
#1	Skeleton and schemas	Typed models, config, CI	Env example, README, schema round-trip tests
#2	Demo app and alert	Model probe, day-one test	Seeded checkout service, red reproducer, invariant catalog, signed webhook, E2B hello world
#3	Baseline and verifier	Baseline and final verifier	Pinned checkout, sandbox commands, patch policy, evidence manifest
#4	Agent-to-PR integration	Nebius patch loop, orchestrator, GitHub PR, progress UI	Post-mortem template, store and events; both run full end-to-end
#5	Discovery and evaluation	Model scenario proposals and optional second repair	Trusted runner, recheck, negative control; CSV and docs; video together

> **Final principle.** The narrative is not “our model always finds bugs.” It is: our agent explores creative repair hypotheses, but trusted, reproducible evidence controls every claim and every PR.